

Experiment 3

Aim

Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Guidelines

Lab Objectives: To implement public and private Blockchain.

Lab Outcomes (LO): Demonstrate the concept of Blockchain in real-world Applications (LO4)

Task to be performed

1. Download the code from folder, Lab_3
2. Install requests in the virtual environment created in the Lab 2. (Follow the instructions)
3. Run the files - hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py in 3 different terminals.
4. Open Postman, from each node - invoke connect_node() and pass the peers as POST requests.
5. Perform the following functions
 - Add Transactions - invoke add_transactions() as a POST request.
 - mining - mine_block(),
 - fetch the chain - get_chain(),
 - replace the longest chain - replace_chain()
6. Modify the code such that transactions are removed after they are added to the block.

Tools & Libraries used

- Install Flask : pip install Flask
- Download Postman from <https://www.postman.com/>
- Python Libraries : datetime, jsonify, hashlib, uuid4, urlparse, request
- Install requests : pip install requests==2.18.4

Instructions

Prepare for viva for the following topics:

- Challenges in P2P networks
- How transactions are performed on the network?
- Explain the role of mempools
- Write briefly about the libraries and the tools used during implementation.

Outcome

1. Understood the challenges in P2P networks, how transactions are performed and how a miner mines a

block to be added in a blockchain.

2. Implemented a Cryptocurrency in Python using Flask, Postman and Python libraries such as datetime, jsonify, hashlib, uuid4, urlparse, request.
3. Successfully mined the blocks among a P2P network with 3 nodes.
4. Performed transactions via the network.
5. Successfully updated the block across the network
6. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.
7. Submit the hardcopy by the 2nd week of August 2023 (As per the instructions, submit a hard copy of the same).

Theory

1. Blockchain Overview

Blockchain is a distributed and decentralized ledger that stores information in a series of linked blocks. Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes immutable because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.

Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate three independent blockchain nodes (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the Longest Chain Rule:

- If multiple versions of the chain exist, the longest valid chain is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A reward transaction is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- Python 3.x
- Flask – Web framework for API endpoints
- Requests – For HTTP communication between nodes
- Postman – For testing API requests
- Python Standard Libraries:

```
datetime  
jsonify  
hashlib  
uuid4  
urlparse  
request
```

Procedure and Output

1. Download hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py.
2. Install required packages.
3. Run each node in separate terminals.

← → ↻ ⓘ 127.0.0.1:5001/get_chain

Pretty print ☒

```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-02 12:30:40.901967",
      "transactions": []
    }
  ],
  "length": 1
}
```

← → ↻ ⓘ 127.0.0.1:5002/get_chain

Pretty print ☒

```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-02 12:32:12.094660",
      "transactions": []
    }
  ],
  "length": 1
}
```

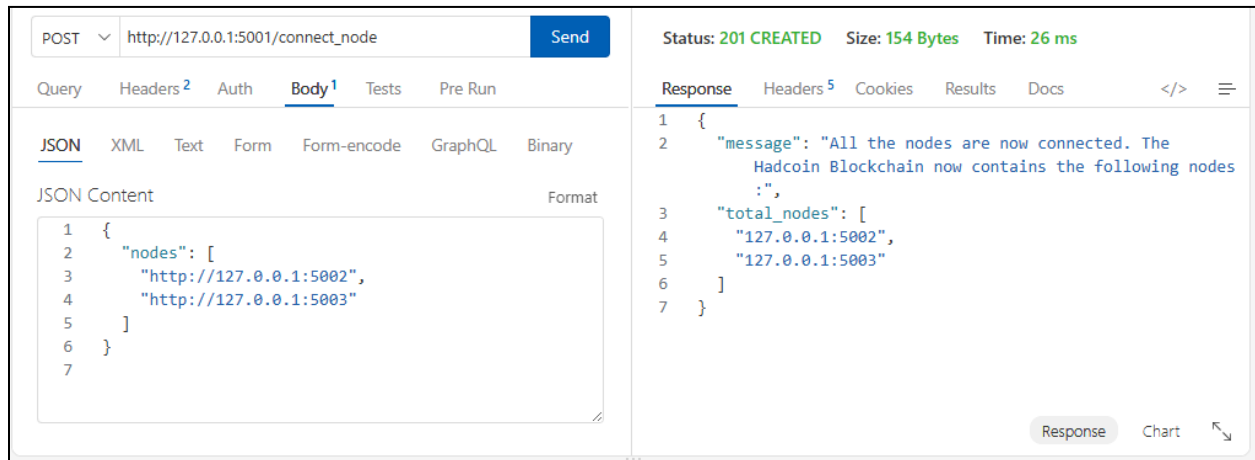
← → ↻ ⓘ 127.0.0.1:5003/get_chain

Pretty print ☒

```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-02 12:36:59.744100",
      "transactions": []
    }
  ],
  "length": 1
}
```

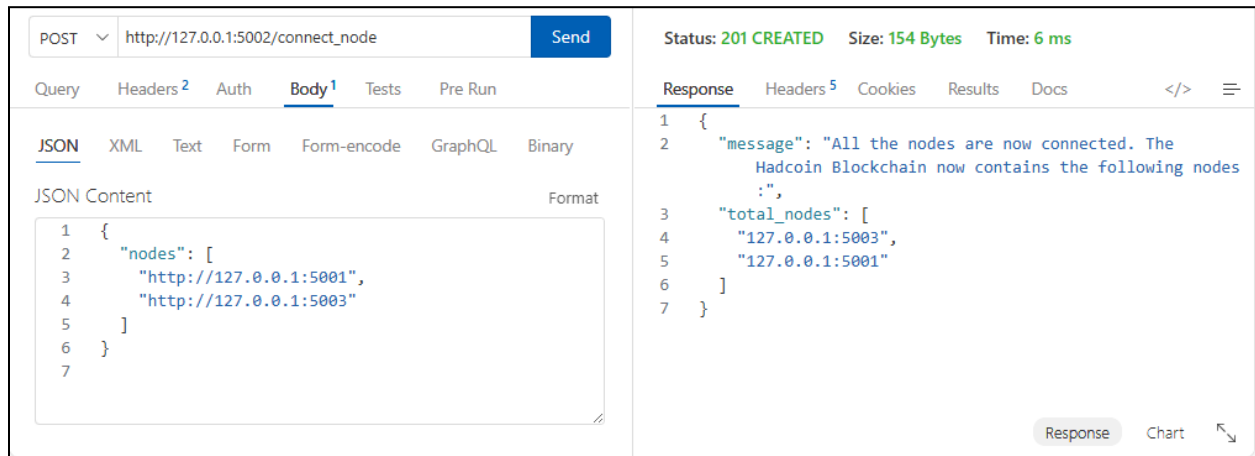
4. Connect nodes using Postman – POST /connect_node.

```
{
  "nodes": [
    "http://127.0.0.1:5002",
    "http://127.0.0.1:5002"
  ]
}
```



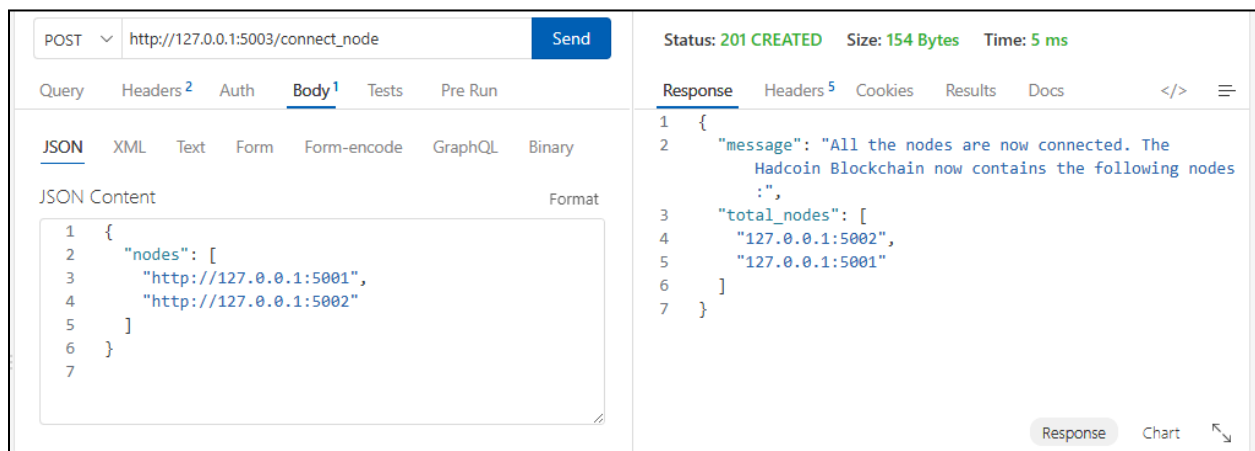
Postman interface showing a POST request to `http://127.0.0.1:5001/connect_node`. The request body is a JSON object with a `nodes` array containing two identical entries: `"http://127.0.0.1:5002"`. The response status is **201 CREATED**, with a size of 154 Bytes and a time of 26 ms. The response body is a JSON object with a `message` field stating "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes :", and a `total_nodes` array containing the two nodes from the request.

```
{
  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes :",
  "total_nodes": [
    "http://127.0.0.1:5002",
    "http://127.0.0.1:5002"
  ]
}
```



Postman interface showing a POST request to `http://127.0.0.1:5002/connect_node`. The request body is a JSON object with a `nodes` array containing two entries: `"http://127.0.0.1:5001"` and `"http://127.0.0.1:5003"`. The response status is **201 CREATED**, with a size of 154 Bytes and a time of 6 ms. The response body is a JSON object with a `message` field stating "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes :", and a `total_nodes` array containing the two nodes from the request.

```
{
  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes :",
  "total_nodes": [
    "http://127.0.0.1:5003",
    "http://127.0.0.1:5001"
  ]
}
```



Postman interface showing a POST request to `http://127.0.0.1:5003/connect_node`. The request body is a JSON object with a `nodes` array containing two entries: `"http://127.0.0.1:5001"` and `"http://127.0.0.1:5002"`. The response status is **201 CREATED**, with a size of 154 Bytes and a time of 5 ms. The response body is a JSON object with a `message` field stating "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes :", and a `total_nodes` array containing the two nodes from the request.

```
{
  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes :",
  "total_nodes": [
    "http://127.0.0.1:5002",
    "http://127.0.0.1:5001"
  ]
}
```

5. Add transactions – POST

POST

http://127.0.0.1:5001/add_transaction

Send

Status: 201 CREATED

Size: 56 Bytes

Time: 5 ms

Query

Headers 2

Auth

Body 1

Tests

Pre Run

Response

Headers 5

Cookies

Results

Docs

</>

≡

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

```

1 {
2   "sender": "Saniya",
3   "receiver": "Manaswi",
4   "amount": 50
5 }
6

```

1 {

2 "message": "This transaction will be added to Block 2"

3 }

Response

Chart

↗

6. Add 3 transactions in 5001

7. Condition 2 — Each node must have different chain lengths

Example

If you mined 1 block on 5001, but did not mine on 5002 or 5003 →

- 5001 chain length = 2
- 5002 chain length = 1
- 5003 chain length = 1

STEP 2 — Perform all transactions ONLY from one node (e.g., 5001)

POST → http://127.0.0.1:5001/add_transaction

```

{
  "sender": "a",
  "receiver": "b",
  "amount": 5
}

```

Then 5001 → GET /mine_block

GET

http://127.0.0.1:5001/mine_block

Send

Status: 200 OK

Size: 298 Bytes

Time: 64 ms

Query

Headers 2

Auth

Body

Tests

Pre Run

Response

Headers 5

Cookies

Results

Docs

</>

≡

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

1 {

2 "index": 3,

3 "message": "Congratulations, you just mined a block!",

4 "previous_hash":

5 "9d80784e965e2f2cb1eddef7395f962e5d49b265b3632ad31

6 "f6f40825aaca0fb",

7 "proof": 45293,

8 "timestamp": "2026-09-02 12:59:03.189390",

9 "transactions": [

10 {

11 "amount": 1,

12 "receiver": "Richard",

13 "sender": "b92cf3fc78bb42f9a22a34d19fd663ae"

14 }

15]

16 }

STEP 3 — Now go to 5002 and run

GET → /replace_chain

STEP 4 — Repeat on 5003

GET → /replace_chain

Same result.

GET

Query Headers² Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

1

Status: 200 OK Size: 502 Bytes Time: 12 ms

Response Headers⁵ Cookies Results Docs

```
1 {
2   "message": "The nodes had different chains so the
3   chain was replaced by the longest one.",
4   "new_chain": [
5     {
6       "index": 1,
7       "previous_hash": "0",
8       "proof": 1,
9       "timestamp": "2026-09-02 12:30:40.901967",
10      "transactions": []
11    },
12    {
13      "index": 2,
14      "previous_hash":
15        "3c1bfa70f4436263095cce35a71c4384471ee07bf93b
16        a38298709ef607a9133c",
17      "proof": 533,
18      "timestamp": "2026-09-02 12:48:27.434419",
19      "transactions": [
20        {
21          "amount": 50,
22          "receiver": "Manaswi",
23          "sender": "Saniya"
24        }
25      ]
26    }
27  ]
28 }
```

GET

Query Headers² Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

1

Status: 200 OK Size: 502 Bytes Time: 12 ms

Response Headers⁵ Cookies Results Docs

```
1 {
2   "message": "The nodes had different chains so the
3   chain was replaced by the longest one.",
4   "new_chain": [
5     {
6       "index": 1,
7       "previous_hash": "0",
8       "proof": 1,
9       "timestamp": "2026-09-02 12:30:40.901967",
10      "transactions": []
11    },
12    {
13      "index": 2,
14      "previous_hash":
15        "3c1bfa70f4436263095cce35a71c4384471ee07bf93b
16        a38298709ef607a9133c",
17      "proof": 533,
18      "timestamp": "2026-09-02 12:48:27.434419",
19      "transactions": [
20        {
21          "amount": 50,
22          "receiver": "Manaswi",
23          "sender": "Saniya"
24        }
25      ]
26    }
27  ]
28 }
```

```

"receiver": "b",
"amount": 53
}

```

Output

```

{
  "index": 3,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "d501f26a58a2bfccd173a1da675769cae717df3b4536e0f2698fe8d27a45436b",
  "proof": 45293,
  "timestamp": "2026-01-28 15:34:33.283321",
  "transactions": [
    {"amount": 51, "receiver": "b", "sender": "a"},
    {"amount": 52, "receiver": "b", "sender": "a"},
    {"amount": 53, "receiver": "b", "sender": "a"},
    {"amount": 1, "receiver": "Richard", "sender": "609febafd5104eafa33aecb4de9ea7b6"}
  ]
}

```

9. Mine blocks – GET /mine_block.

The screenshot shows a web browser interface with a GET request to `http://127.0.0.1:5001/mine_block` and its corresponding JSON response. The response status is 200 OK, with a size of 349 Bytes and a time of 5 ms. The JSON content is displayed in the 'Body' tab, showing a block with index 2, a message, a previous hash, a proof, a timestamp, and a list of transactions.

```

1 {
2   "index": 2,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash":
5     "3c1bfa70f4436263095cce35a71c4384471ee07bf93ba3829
6     8709ef607a9133c",
7   "proof": 533,
8   "timestamp": "2026-09-02 12:48:27.434419",
9   "transactions": [
10     {
11       "amount": 50,
12       "receiver": "Manaswi",
13       "sender": "Saniya"
14     },
15     {
16       "amount": 1,
17       "receiver": "Richard",
18       "sender": "b92cf3fc78bb42f9a22a34d19fd663ae"
19     }
20   ]
21 }

```

10. Fetch blockchain – GET /get_chain.

GET

http://127.0.0.1:5001/get_chain

Send

Query

Headers²

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

Status: 200 OK

Size: 420 Bytes

Time: 4 ms

Response

Headers⁵

Cookies

Results

Docs

</>

≡

8

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

"transactions": []

,

{

"index": 2,

"previous_hash":

"3c1bfa70f4436263095cce35a71c4384471ee07bf93b

a38298709ef607a9133c",

"proof": 533,

"timestamp": "2026-09-02 12:48:27.434419",

"transactions": [

{

"amount": 50,

"receiver": "Manaswi",

"sender": "Saniya"

},

{

"amount": 1,

"receiver": "Richard",

"sender": "b92cf3fc78bb42f9a22a34d19fd663ae"

}

]

}

,

"length": 2

}

GET

http://127.0.0.1:5001/get_chain

Send

Query

Headers²

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

Status: 200 OK

Size: 665 Bytes

Time: 3 ms

Response

Headers⁵

Cookies

Results

Docs

</>

≡

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

{

"amount": 1,

"receiver": "Richard",

"sender": "b92cf3fc78bb42f9a22a34d19fd663ae"

}

]

,

{

"index": 3,

"previous_hash":

"9d80784e965e2f2cb1eddef7395f962e5d49b265b363

2ad31f6f40825aaca0fb",

"proof": 45293,

"timestamp": "2026-09-02 12:59:03.189390",

"transactions": [

{

"amount": 1,

"receiver": "Richard",

"sender": "b92cf3fc78bb42f9a22a34d19fd663ae"

}

]

}

,

"length": 3

}

GET

http://127.0.0.1:5002/get_chain

Send

Status: 200 OK

Size: 124 Bytes

Time: 4 ms

Query

Headers 2

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

Response

Headers 5

Cookies

Results

Docs

</>

≡

```

1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-02 12:32:12.094660",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }

```

GET

http://127.0.0.1:5003/get_chain

Send

Status: 200 OK

Size: 420 Bytes

Time: 3 ms

Query

Headers 2

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

Response

Headers 5

Cookies

Results

Docs

</>

≡

```

8   "transactions": []
9 },
10 {
11   "index": 2,
12   "previous_hash":
13     "3c1bfa70f4436263095cce35a71c4384471ee07bf93b
14     a38298709ef607a9133c",
15   "proof": 533,
16   "timestamp": "2026-09-02 12:48:27.434419",
17   "transactions": [
18     {
19       "amount": 50,
20       "receiver": "Manaswi",
21       "sender": "Saniya"
22     },
23     {
24       "amount": 1,
25       "receiver": "Richard",
26       "sender": "b92cf3fc78bb42f9a22a34d19fd663ae"
27     }
28   ]
29 },
30 "length": 2

```

11. Valid chain – GET /is_valid.

GET

http://127.0.0.1:5001/is_valid

Send

Status: 200 OK

Size: 49 Bytes

Time: 3 ms

Query

Headers 2

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

Response

Headers 5

Cookies

Results

Docs

</>

≡

```

1 {
2   "message": "All good. The Blockchain is valid."
3 }

```

Conclusion

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.